

Here is the comprehensive, strictly syllabus-bound Java Full Stack Question Bank based entirely on the topics you have completed so far.

JAVA FULL STACK QUESTION BANK

1. Java Fundamentals

A. Basic Theory

1. [Level 1] What are the core differences between the JDK, JRE, and JVM?
2. [Level 1] Define platform independence in the context of Java. What makes it possible?
3. [Level 1] What is the purpose of setting the JAVA_HOME and PATH environment variables?

B. Conceptual

1. [Level 2] Explain the complete journey of a Java program from a .java file to execution by the CPU.
2. [Level 2] Why is Java considered both a compiled and an interpreted language?
3. [Level 2] How does the Just-In-Time (JIT) compiler improve the performance of a Java application compared to pure interpretation?

C. Programming

1. [Level 1] Write the exact terminal commands required to compile a file named App.java and then execute the resulting compiled class.
2. [Level 2] Write a basic JShell sequence of commands to declare a variable and print it without creating a formal class or main() method.
3. [Level 3] *Skipped - Programming scope is limited for purely theoretical environment setup.*

D. Output-Based

1. [Level 1] *Skipped - Not applicable for theoretical fundamentals.*
2. [Level 2] *Skipped*
3. [Level 3] *Skipped*

E. Debugging

1. [Level 1] You type javac MyApp.java in the terminal and receive the error: 'javac' is not recognized as an internal or external command. What is the issue and how do you fix it?
2. [Level 2] You successfully compiled MyApp.java into MyApp.class. You type java MyApp.class in the terminal and get an error: Could not find or load main class MyApp.class. Why does this happen?
3. [Level 3] You have JDK 21 installed, but an older enterprise application requires Java 8 to run. How do you manage this conflict using environment variables?

F. Code Tracing

1. [Level 2] Trace the behavior of the Classloader, Bytecode Verifier, and Execution Engine inside the JVM architecture when a .class file is executed.
2. [Level 2] *Skipped - limited scope.*
3. [Level 2] *Skipped - limited scope.*

G. Comparison

1. [Level 1] Java vs C/C++: Why does Java not support explicit pointers?
2. [Level 1] Java SE vs Java EE (Jakarta EE): What is the primary difference in their use cases?
3. [Level 2] JDK vs JRE: If you are only deploying a finished application to a production server, which one do you install and why?

H. Scenario-Based

1. [Level 2] You are distributing your Java application to a client. They are not developers, they just want to run the app. What exact software components must they install on their machine?
2. [Level 3] You develop a Java application on a Windows machine. Your server runs Linux, and your client uses a Mac. Explain exactly why you do not need to recompile the code for each operating system.
3. [Level 3] You are working in a highly constrained environment with very limited memory. Based on the JVM architecture, what challenges might you face?

I. Interview

1. [Level 4 - Beginner] "Java is write once, run anywhere." Explain how the JVM makes this true.
2. [Level 4 - Intermediate] Can you run a Java program without a main() method in modern Java (JDK 21+)? (Hint: Think about JShell or unnamed classes).
3. [Level 4 - Advanced] Explain how the JIT compiler identifies "hotspots" in the code.

J. Advanced / Tricky

1. [Level 3] Is bytecode machine code? If not, what actually reads the bytecode?
2. [Level 3] If a compiled .class file is transferred to a machine with a different architecture (e.g., ARM vs x86), does the bytecode change?
3. [Level 4] Why does Java have a Bytecode Verifier? What security threat does it prevent?

2. Java Syntax & Basics

A. Basic Theory

1. [Level 1] What are the 8 primitive data types in Java? Provide their sizes in bytes.
2. [Level 1] What is the difference between a local variable, an instance variable, and a static variable?
3. [Level 1] What does the final keyword do when applied to a variable?

B. Conceptual

1. [Level 2] Why are Strings considered non-primitive data types in Java, and how does their memory allocation differ from an int?
2. [Level 2] Explain the concept of Type Promotion in Java expressions. Why does Java automatically promote byte and short to int during arithmetic?
3. [Level 3] Why is explicit casting required for narrowing conversions (e.g., double to int), but not for widening conversions (e.g., int to double)?

C. Programming

1. [Level 1 - Easy] Write a program that declares a final constant for the value of PI,

calculates the area of a circle for a given radius, and prints the result.

2. [Level 2 - Medium] Write a program that demonstrates explicit type casting by converting a double value to an int and a char to an int.
3. [Level 3 - Hard] Write a program that declares an instance variable and a local variable with the exact same identifier name. Demonstrate how to print both values.

D. Output-Based

1. [Level 2] What is the output?

```
Java
byte b1 = 10;
byte b2 = 20;
byte b3 = b1 + b2;
System.out.println(b3);
```

2. [Level 3] What is the output?

```
Java
char c = 'A';
int val = c + 1;
System.out.println((char)val);
```

3. [Level 3] What is the output?

```
Java
int a = 130;
byte b = (byte) a;
System.out.println(b);
```

E. Debugging

1. [Level 1] Why does this code cause a compilation error, and how do you fix it?

```
Java
float f = 3.14;
```

2. [Level 2] Identify the error:

```
Java
final int MAX_SPEED;
System.out.println(MAX_SPEED);
```

3. [Level 2] Identify the logical error:

```
Java
int x = 5;
int y = 2;
double result = x / y;
System.out.println("Result: " + result);
```

F. Code Tracing

1. [Level 2] Trace the memory allocation (Stack vs Heap concept, implicitly based on primitives vs objects) for: `int age = 25; String name = "Alice";`.
2. [Level 3] Trace the exact values of variables step-by-step:
Java
`double d = 100.04;`
`long l = (long) d;`
`int i = (int) l;`
3. [Level 2] Trace the initialization lifecycle of a static variable vs an instance variable when a class is loaded and an object is created.

G. Comparison

1. [Level 1] `int` vs `Integer` (Primitive vs Non-primitive wrappers - based on data type rules).
2. [Level 2] Local variables vs Instance variables (Default values and scope constraints).
3. [Level 3] Explicit casting vs Implicit type conversion.

H. Scenario-Based

1. [Level 2] You are storing the exact price of a product in a financial application. Should you use `float`, `double`, or something else? Why are primitive floating points dangerous for currency?
2. [Level 2] You need a variable to represent the maximum number of players in a game server. This number must be the same for all objects of the `Server` class and must never change. How do you declare it?
3. [Level 3] You receive an integer from an external API representing a file size in bytes, which could be up to 5 Gigabytes. Which primitive type must you use to store this without data loss?

I. Interview

1. [Level 4 - Beginner] What are valid and invalid identifiers in Java? Can an identifier start with `$` or `_`?
2. [Level 4 - Intermediate] Why does Java have both primitives and objects? Why not make everything an object like in Python?
3. [Level 4 - Advanced] What happens internally during a narrowing conversion when the value exceeds the capacity of the target data type (e.g., casting 130 to `byte`)?

J. Advanced / Tricky

1. [Level 3] Can you declare a main method as `final`?
2. [Level 3] What happens if you define a local variable but never initialize it, and then try to return it?
3. [Level 4] Why is `String` explicitly designed as a non-primitive, even though it acts similarly to a primitive in basic syntax?

3. Operators

A. Basic Theory

1. [Level 1] What is the difference between `=` and `==`?
2. [Level 1] Explain the purpose of the Modulo (`%`) operator with an example.

3. [Level 1] What is operator precedence, and why does Java need it?

B. Conceptual

1. [Level 2] Explain Short-circuit evaluation. Why is it an optimization technique?
2. [Level 2] How does the Bitwise AND (&) differ from the Logical AND (&&) internally?
3. [Level 3] What is the precise execution difference between the prefix (++x) and postfix (x++) unary operators?

C. Programming

1. [Level 1 - Easy] Write a program to check if a number is even or odd using the Modulo operator.
2. [Level 2 - Medium] Write a program that uses the Ternary operator to find the maximum of three numbers.
3. [Level 3 - Hard] Write a program to swap two integer variables without using a temporary third variable, utilizing the Bitwise XOR (^) operator.

D. Output-Based

1. [Level 2] What is the output?

```
Java
int x = 5;
int y = x++ + ++x;
System.out.println(y);
```

2. [Level 2] What is the output?

```
Java
boolean a = true;
boolean b = false;
boolean c = a || (b = true);
System.out.println(b);
```

3. [Level 3] What is the output?

```
Java
int a = -10;
int b = a >> 1;
int c = a >>> 1;
System.out.println(b + " " + (c > b));
```

E. Debugging

1. [Level 1] Identify the bug: `if (score = 100) { System.out.println("Perfect!"); }`
2. [Level 2] Identify the bug:

```
Java
int x = 10;
x = x++;
System.out.println(x);
```

3. [Level 3] Identify the conceptual bug: `boolean valid = (obj != null) & (obj instanceof String);`

F. Code Tracing

1. [Level 2] Dry run the expression using operator precedence: `int result = 5 + 3 * 2 % 4 - 1;`
2. [Level 3] Trace the bitwise logic for: `int z = 5 & 3;` (Show binary conversion).
3. [Level 3] Trace the execution of: `String status = (age > 18) ? (hasTicket ? "Enter VIP" : "Enter General") : "No Entry";`

G. Comparison

1. [Level 1] Logical AND (&&) vs Logical OR (||).
2. [Level 2] Right Shift (>>) vs Unsigned Right Shift (>>>).
3. [Level 2] Bitwise OR (|) vs Logical OR (||) when dealing with boolean variables.

H. Scenario-Based

1. [Level 2] You are verifying a user login. You must check if the database connection is active, AND if the credentials match. Why is it critical to use && instead of & here?
2. [Level 3] You need to multiply an integer by exactly 8 as fast as possible for a high-performance graphics engine. Which operator should you use instead of *?
3. [Level 3] You want to quickly check if a given object is a String before attempting to cast it. Which operator is designed specifically for this?

I. Interview

1. [Level 4 - Beginner] What is the associativity of the Assignment operator (=)? (Left-to-Right or Right-to-Left?)
2. [Level 4 - Intermediate] Can you apply the modulus operator (%) to floating-point numbers in Java?
3. [Level 4 - Advanced] Explain how instanceof handles null values. Does null instanceof String return true, false, or throw an exception?

J. Advanced / Tricky

1. [Level 3] What happens if you do `int x = 10 / 0;`? What happens if you do `double x = 10.0 / 0;`?
2. [Level 4] Why does `System.out.println(10 + 20 + "Java");` output 30Java while `System.out.println("Java" + 10 + 20);` outputs Java1020?
3. [Level 4] Evaluate: `int i = ~10;`. What is the value of i?

4. Control Flow

A. Basic Theory

1. [Level 1] What is the difference between a while loop and a do-while loop?
2. [Level 1] Explain the role of the break and continue keywords in a loop.
3. [Level 1] What are the components of a standard for loop declaration?

B. Conceptual

1. [Level 2] Why were Modern Switch expressions and the yield keyword introduced in newer versions of Java? What traditional switch problem do they solve?
2. [Level 2] When using an else-if ladder, why does the order of conditions matter drastically?
3. [Level 3] Why is it dangerous to use floating-point numbers (float/double) as loop counters?

C. Programming

1. [Level 1 - Easy] Write a program using a for loop to print the multiplication table of a given number.
2. [Level 2 - Medium] Write a program using a Modern Switch Expression to take an integer (1-7) and return whether it is a "Weekday" or "Weekend".
3. [Level 3 - Hard] Write a program using nested loops to print a pyramid pattern of asterisks (*).

D. Output-Based

1. [Level 2] What is the output?

```
Java
for (int i = 0; i < 5; i++) {
    if (i == 3) continue;
    System.out.print(i + " ");
}
```

2. [Level 3] What is the output?

```
Java
int day = 2;
switch(day) {
    case 1: System.out.print("A");
    case 2: System.out.print("B");
    case 3: System.out.print("C"); break;
    default: System.out.print("D");
}
```

3. [Level 3] What is the output?

```
Java
int x = 0;
while (x++ < 3) {
    System.out.print(x + " ");
}
```

E. Debugging

1. [Level 1] Identify the infinite loop bug:

```
Java
int count = 5;
while (count > 0) {
    System.out.println(count);
}
```

2. [Level 2] Identify the compilation error:

```
Java
int[] arr = {1, 2, 3};
```

```

for (int num : arr) {
    num = num * 2;
}
// Expecting arr to be {2, 4, 6}

```

3. [Level 3] Identify the logic bug in this modern switch using yield:

```

Java
int score = 10;
String result = switch (score) {
    case 10 -> { System.out.println("Perfect"); }
    default -> "Average";
};

```

F. Code Tracing

1. [Level 2] Dry run a nested loop where the outer loop runs 3 times and the inner loop runs 4 times. How many total iterations occur?
2. [Level 3] Trace the exact step-by-step execution path of an if-else block where an inner nested if contains a return statement.
3. [Level 2] Trace the execution of a do-while loop where the initial condition evaluates to false on the very first check.

G. Comparison

1. [Level 1] for loop vs while loop: When should you use which?
2. [Level 2] Standard switch statement vs Modern Switch expression (->).
3. [Level 2] break vs return inside a loop structure.

H. Scenario-Based

1. [Level 2] You are writing a script to read incoming network packets. You don't know how many packets will arrive, but you must stop reading if a packet contains the word "END". Which loop is best for this?
2. [Level 2] You are iterating through an array of user objects. You need to read every single user without modifying them or skipping any. Which loop structure is the safest and cleanest to use?
3. [Level 3] You have a massive else-if ladder checking 20 different exact String matches. For performance and readability, what control flow structure should you refactor this into?

I. Interview

1. [Level 4 - Beginner] What data types are allowed to be evaluated inside a traditional switch statement in Java?
2. [Level 4 - Intermediate] Can you write a standard for loop with multiple initialization variables and multiple update statements? (e.g., for(int i=0, j=10; i<j; i++, j--))
3. [Level 4 - Advanced] In a Modern Switch expression, what happens if you do not cover all possible values and omit the default case?

J. Advanced / Tricky

1. [Level 3] Can you put a break statement inside an if block that is *not* inside a loop or

switch?

2. [Level 4] Is it possible to have an empty for loop declaration like for(;;)? What does it do?
3. [Level 4] What is the difference between break in a traditional switch vs yield in a switch expression?

5. Methods

A. Basic Theory

1. [Level 1] What is a method signature in Java?
2. [Level 1] Explain the difference between method parameters and arguments.
3. [Level 1] What does the void return type indicate?

B. Conceptual

1. [Level 2] Java is strictly "Pass-by-Value". Explain what this means when passing primitive types versus passing Object references to a method.
2. [Level 2] What is Method Overloading? How does the compiler know which overloaded method to execute?
3. [Level 3] Explain the concept of Recursion. Why is a "base case" absolutely mandatory?

C. Programming

1. [Level 1 - Easy] Write a method named calculateArea that takes the length and width of a rectangle as parameters and returns the area.
2. [Level 2 - Medium] Write a recursive method to calculate the factorial of a given integer.
3. [Level 3 - Hard] Write an overloaded method add() that can take either two integers, three integers, or an array of integers (using Varargs) and return their sum.

D. Output-Based

1. [Level 2] What is the output?

Java

```
public static void modify(int x) { x = 100; }  
public static void main(String[] args) {  
    int a = 10;  
    modify(a);  
    System.out.println(a);  
}
```

2. [Level 3] What is the output?

Java

```
public static void modifyArr(int[] arr) { arr[0] = 99; }  
public static void main(String[] args) {  
    int[] nums = {1, 2, 3};  
    modifyArr(nums);  
    System.out.println(nums[0]);  
}
```

3. [Level 3] What is the output of mystery(3)?

```

Java
int mystery(int n) {
    if (n == 0) return 1;
    return n * mystery(n - 1);
}

```

E. Debugging

1. [Level 1] Identify the error:

```

Java
public int getNumber() {
    System.out.println("Returning 5");
    // Missing something here
}

```

2. [Level 2] Identify the compilation error regarding Varargs:

```

Java
public void printDetails(String... names, int age) { }

```

3. [Level 3] Identify the flaw in this recursive method:

```

Java
public void runForever(int n) {
    if (n == 10) return;
    System.out.println(n);
    runForever(n - 1);
}
// Called with runForever(5)

```

F. Code Tracing

1. [Level 2] Trace the execution stack (Call Stack) when a recursive method calls itself 3 times before hitting the base case.
2. [Level 3] Trace memory (heap vs stack) when an Object is passed into a method, modified inside the method, and then the method ends.
3. [Level 3] Trace Method Chaining execution: `myObj.setValue(10).multiply(2).print();`

G. Comparison

1. [Level 1] Static methods vs Instance methods.
2. [Level 2] Method Overloading vs Duplicate Methods.
3. [Level 3] Recursion vs Iteration (Loops): What are the performance and memory trade-offs?

H. Scenario-Based

1. [Level 2] You are writing a utility class `MathHelper` that provides general math formulas (like `findHypotenuse`). Should these methods be declared as static or instance methods? Why?
2. [Level 2] You want to create a logger method that can accept any number of String

messages to print. How should you define the parameters?

3. [Level 3] You need to traverse a nested folder structure on a hard drive where folders can contain folders infinitely. Should you use a standard loop or recursion? Why?

I. Interview

1. [Level 4 - Beginner] Can you overload a method by changing only its return type?
2. [Level 4 - Intermediate] If Java doesn't support pass-by-reference, how is a method able to change the data inside an Array passed to it?
3. [Level 4 - Advanced] What is a StackOverflowError, and how is it caused by recursion?

J. Advanced / Tricky

1. [Level 3] Can a static method call an instance method from the same class without creating an object?
2. [Level 4] If you have two overloaded methods: print(int x) and print(double x), and you call print(5). Which one is executed? What if you call print(5.0)? What if you have print(short x) and call print(5)?
3. [Level 4] Does varargs int... args allow you to pass a pre-existing int[] array as an argument?

CROSS-TOPIC QUESTIONS

1. **[Data Types + Methods + Operators]** Write a method that takes a primitive double, explicitly casts it to an int, applies a bitwise shift operator to it, and returns the result. (Tests type conversion + operators + method syntax).
2. **[Control Flow + Operators]** How can you use the short-circuit OR operator (||) inside an if statement to prevent a "divide by zero" runtime error? (e.g., if(y == 0 || x / y > 10)).
3. **[Methods + Control Flow]** Rewrite a given for loop that calculates the sum of an array into a purely recursive method.
4. **[Fundamentals + Data Types + Control Flow]** You run a Java application that requires a command-line argument. The program takes the String argument, converts it to an int, and passes it into a switch expression. What data types are involved in this lifecycle, and how does the JVM handle the starting main method arguments?
5. **[Operators + Loops]** Dry run the following code: int x = 0; while(x++ < 5) { if(x % 2 == 0) continue; System.out.print(x); } (Tests postfix unary + relational + modulo + loop + continue statement).

MOST IMPORTANT QUESTIONS

● Must Know (Fundamentals & Deal-Breakers)

- Explain JDK vs JRE vs JVM.
- What are the 8 primitive data types?
- Explain the difference between = and ==.
- How does an if-else ladder execute?
- What is the difference between a for loop and a while loop?
- Explain Parameters vs Arguments and void vs Return types.

- What is the break vs continue behavior inside loops?

- When should you use Varargs (...)?

- comparisons or loop counting.

COVERAGE CHECKLIST

Topic	Theor y	Conc eptual	Codin g	Outp ut	Debu gging	Scena rio	Interv iew	Advan ced
1. Funda menta ls	 Covered	 Covered	 Limited scope	 Not applicable	 Covered	 Covered	 Covered	 Covered
2. Synta x & Basics	 Covered	 Covered	 Covered	 Covered	 Covered	 Covered	 Covered	 Covered
3. Opera tors	 Covered	 Covered	 Covered	 Covered	 Covered	 Covered	 Covered	 Covered

4. Contr ol Flow	 Cover ed	 Cover ed	 Cover ed	 Cover ed	 Cover ed	 Cover ed	 Cover ed	 Cover ed
5. Meth ods	 Cover ed	 Cover ed	 Cover ed	 Cover ed	 Cover ed	 Cover ed	 Cover ed	 Cover ed